LAB 3 ASSIGNMENT

Building RESTful Web Services

Express.js • Spring Boot • CRUD • OpenAPI / Swagger

Important: Students must learn both Express.js and Spring Boot. Implement the complete Student REST API in **one** framework, then implement/compare at least two equivalent endpoints (GET and POST) in the other framework. Test with Postman and document with OpenAPI / Swagger.

1. Objective

Build a RESTful Student Management Web Service. The API must expose resource-oriented endpoints, implement CRUD operations, return appropriate HTTP status codes, handle invalid requests, and provide interactive API documentation. Students will understand equivalent REST implementations in both Express.js and Spring Boot.

2. Lab Environment and Project Setup

Common tools: Node.js and npm • Java/JDK • Maven • VS Code or preferred IDE • Postman • Git/GitHub

Express.js	Spring Boot
mkdir student-api cd student-api npm init -y npm install express	Use Spring Initializr with Maven + Java + Spring Web + Validation. Generate the project and open it in the IDE.

3. Resource to Implement

Create a **Student** resource with these fields: **id, name, email, course, semester**. Use JSON for request and response bodies. For Lab 3, an in-memory data structure is sufficient; database integration is reserved for the subsequent lab.

4. API Endpoints

Operation	HTTP Method	Endpoint	Expected Result
List students	GET	/students	200 OK
Get one	GET	/students/{id}	200 OK / 404
Create	POST	/students	201 Created
Update	PUT / PATCH	/students/{id}	200 OK / 404
Delete	DELETE	/students/{id}	204 / 200 OK

5. Learning Both Frameworks — Implementation Strategy

Part A — Express.js: Understand and implement the complete CRUD API.

Part B — Spring Boot: Understand the equivalent Controller → Service → Repository architecture and implement at least two equivalent endpoints, such as GET and POST.

Principle: REST is the API/architectural paradigm; Express.js and Spring Boot are alternative implementation technologies.

6. Step-by-Step Procedure

Step 1 — Set up the environment: Prepare the common tools and initialize both project types.

Step 2 — Design the resource: Define the Student JSON structure and endpoint list before coding.

Step 3 — Implement Express.js CRUD: Implement GET all, GET by ID, POST, PUT/PATCH and DELETE.

Step 4 — Implement Spring Boot equivalents: Trace Controller → Service → Repository and implement GET + POST equivalents.

Step 5 — Validate requests: Reject invalid or incomplete input with HTTP 400 and a structured JSON error.

Step 6 — Test: Verify valid and invalid requests with Postman.

Step 7 — Document: Configure OpenAPI/Swagger and verify the generated Swagger UI.

LAB 3 ASSIGNMENT

Building RESTful Web Services — Practical Implementation

7. Express.js Complete CRUD Requirement

At least one complete CRUD implementation must be demonstrated. A minimal implementation should contain **server.js**, **express.json()**, and these routes: GET /students, GET /students/:id, POST /students, PUT /students/:id, DELETE /students/:id.

Example route behavior: GET by ID returns **200 OK** with the student when found and **404 Not Found** with a JSON message when the ID does not exist.

8. Spring Boot Practical Example

Use the structure **Student** → **StudentController** → **StudentService** → **StudentRepository**. Demonstrate how one or two HTTP endpoints travel from the controller through the service to the repository/in-memory data structure.

Example: GET /students → Controller receives request → Service retrieves students → Repository/in-memory store returns data → Controller returns 200 OK.

Example: POST /students → Controller validates JSON → Service creates the resource → Repository/in-memory store saves it → Controller returns 201 Created.

9. Validation and Error Handling

Invalid request example:

```
{
"name": "",
"email": "invalid-email",
"semester": -2
}
```

Expected: HTTP 400 Bad Request plus a structured JSON error response. Students must test both valid and invalid requests through Postman.

Status	Use
400 Bad Request	Invalid or incomplete request body
404 Not Found	Requested student ID does not exist
500 Internal Server Error	Unexpected server-side failure

10. Postman Testing

• Create a Postman collection named **RESTful Web Services – Lab 3**.

• Test GET all, GET by ID, POST, PUT/PATCH and DELETE.

• Test at least one invalid ID and one invalid request body.

• Capture screenshots showing the request URL/method and response status/body.

11. OpenAPI / Swagger — Practical Documentation

• Configure OpenAPI for the API title/version and Student schema.

• Document every Student endpoint, HTTP method, path parameter, request/response schema and common error responses.

• Open the generated Swagger UI URL (or equivalent OpenAPI viewer).

• Use **Try it out** on at least one endpoint and inspect the status code and JSON response.

• Include a small OpenAPI YAML/JSON example in the documentation or presentation.

Intended flow: REST API → OpenAPI Specification → Swagger UI → Interactive API Testing

12. API Design Exercise

Resource	Endpoint	Method	Request Body	Success	Errors
Student	/students	GET	—	200	500
Student	/students/{id}	GET	—	200	404
Student	/students	POST	Student JSON	201	400
Student	/students/{id}	PUT/PATCH	Student JSON	200	400, 404
Student	/students/{id}	DELETE	—	204/200	404

Explain in a few lines why the endpoints are resource-oriented and why the selected HTTP methods/status codes are appropriate.

LAB 3 ASSIGNMENT

Submission and Evidence

13. Submission Requirements

• Working Express.js or Spring Boot project with complete Student CRUD.

• Evidence that both frameworks were learned: complete CRUD in the selected stack plus equivalent GET and POST implementation/trace in the other stack.

• Validation and basic error handling, including an invalid request returning HTTP 400.

• Postman screenshots or exported collection showing all CRUD requests plus negative tests.

• OpenAPI specification and Swagger UI screenshots showing interactive documentation.

• README.md with setup/run commands, endpoint mapping and sample JSON.

• Completed API-design exercise.

14. Suggested Sample Request

```
POST /students
Content-Type: application/json
```

```
{
"name": "Aarav Patel",
"email": "aarav@example.com",
"course": "Computer Science",
"semester": 5
}
```

15. Learning Outcome

By the end of the lab, students should be able to design, implement, test and document a resource-oriented RESTful Web Service; implement CRUD operations using Express.js or Spring Boot; understand equivalent implementation in the other framework; validate requests and handle errors; test APIs using Postman; and document APIs using OpenAPI/Swagger.

Code	Testing	Documentation	API Design
■ Working project	■ Postman CRUD evidence	■ OpenAPI / Swagger	■ Resources + endpoints
■ Complete CRUD in selected ■ stack	■ Invalid ID test	■ Swagger UI screenshots	■ HTTP methods
■ Equivalent GET + POST in ■ other stack	■ Invalid body test	■ README	■ Request / response
■ Validation	■ Positive + negative tests	■ Setup / endpoint mapping	■ Status codes
■ Error handling			

LAB 3 ASSIGNMENT

Scope and Implementation Notes

17. Framework Comparison

REST concern	Express.js	Spring Boot
Routing	Express route handlers	@GetMapping / @PostMapping / @PutMapping / @DeleteMapping
Request JSON	express.json()	@RequestBody
Validation	Explicit checks / validation library	Spring Validation annotations
Business layer	Route/service functions as appropriate	Controller → Service → Repository
Testing	Postman	Postman
Documentation	OpenAPI/Swagger tooling	OpenAPI/Swagger tooling

18. Important Lab Scope Note

Do not add MongoDB/Firebase or other database integration to Lab 3. The intended progression is **Lab 3: REST API + in-memory data**, followed by **Lab 4: REST API + MongoDB/Firebase**.

19. Core Principle to Remember

Students learn both frameworks, but the complete project is implemented in one framework to keep the laboratory feasible. The second framework is used to understand and demonstrate equivalent REST behavior. This preserves the learning objective of exposure to both major technology stacks.

20. Expected Flow

REST Concepts → API Design → Express.js Implementation → Spring Boot Implementation → Validation & Error Handling → Postman Testing → OpenAPI/Swagger → Lab Assignment